

No.1 DesignAI Team

Survivor

Team Number: 14

**Instructions:*

1. **Directly** work on your report in this Google doc. Do not work elsewhere and paste it here.
2. Use the template formatting, if it is a heading (e.g., Heading 1, Heading 2 etc.), format it as such so the **Table of Contents** will reflect it.
3. The point of this Google doc is to be a **live** document that you continually update as you work on your 1D project. The teaching staff will be taking a look at these documents as you work on them to see how your team is progressing.

Members:

Zhang Xirui Asher 1009287 (Treasurer)

Yip Jing Han 1008936

Kwok Zhan Rui 1008758

Ong Yu Hang 1008750

Lin Chao Ran 1008938

Rajeev Bharath 1009193

Chan Yong Ze 1009045

Table of Contents

[Table of Contents](#)

[Introduction](#)

[Game Design](#)

[Description of the Game](#)

[User Manual](#)

[Design Inspirations](#)

[Electronic Design](#)

[Input/Output](#)

[Datapath](#)

[FSM](#)

[Budget](#)

[2D](#)

[Sustainability](#)

[Summary](#)

[References](#)

[Appendix](#)

[1. ALU Design and Tests](#)

[2. Prototype code + Repo link](#)

[3. Project Management Log: Team Tasks](#)

[4. Components' Specifications](#)

Introduction

This project is a grid-based survival game played on a 16x16 board. The player must avoid enemies, manage limited ammo and collect ammo while surviving as long as possible. This game focuses on resource management under pressure.

Design Inspirations

When looking through seniors' past projects, we were inspired by the retro-styled games that most of them created, as well as the simple grid-based layouts that many of them used.

For the gameplay, we wanted to create a top-down survival shooter. We were inspired by the "survivors-like" genre popularized by *Vampire Survivors* (2022) and its many spin-offs like *Holocure* (2023). We want to bring high-adrenaline gameplay, requiring reflexes for movement and combat as well as elements of resource management.

We were fully convinced when one of our groupmates (Jing Han) coded a mockup Python terminal game and everyone loved it. We were huddled around the computer, taking turns playing it and saying "last round" whenever we started the next round after dying.

Game Design

Description of the Game

This game is a **top-down survival shooter** played on a 16 x 16 grid. The objective is to survive for 60 seconds while eliminating as many enemies as possible to gain points.

The game requires movement, strategy, timing and resource management.

Enemies spawn endlessly and move towards the player, forcing the player to dodge and weave through enemies using fast and precise movement.

To put more pressure on the player, the player has limited ammo, which must be replenished by ammo pickups. Moving to the next ammo pickup serves as a constant short-term objective and forces players to constantly move around rather than stay in the same location.

In order to survive, players must use strategies like rounding or lining up enemies to eliminate as many as possible with each attack. To gain the most points, players should avoid allowing enemies to merge since merged enemies only count as one elimination and only give one point.

The game is fun and challenging at all skill levels. New players usually fail to eliminate enemies or collect ammo fast enough, and get cornered or run out of ammo. On the other hand, skilled players tend to take unnecessary risks in order to conserve ammo or gain points, and end up losing due to carelessness.

User Manual

Buttons

1 Start button, 4 Move buttons (left side), 4 Attack buttons (right side)

Screen navigation:

Start screen: Press start to enter difficulty selection.

Difficulty select: Press the left and right **move** buttons to switch difficulties (EASY, MEDIUM, HARD) and press **start** to enter weapon selection.

Weapon select: Press the left and right **move** buttons to switch weapons (LASER, BLAST, SLASH) and press **start** to start the game.

Win/lose screen: Press start to return to the start screen.

Difficulties:

Difficulty	Enemy spawn rate	Enemy move rate
Easy	0.5Hz	0.5Hz
Medium	1Hz	1Hz
Hard	3Hz	1Hz

Gameplay:

Move around, avoid enemies and collect ammo pickups. Attack enemies using the attack buttons. Survive for 60 seconds while eliminating as many enemies as possible.

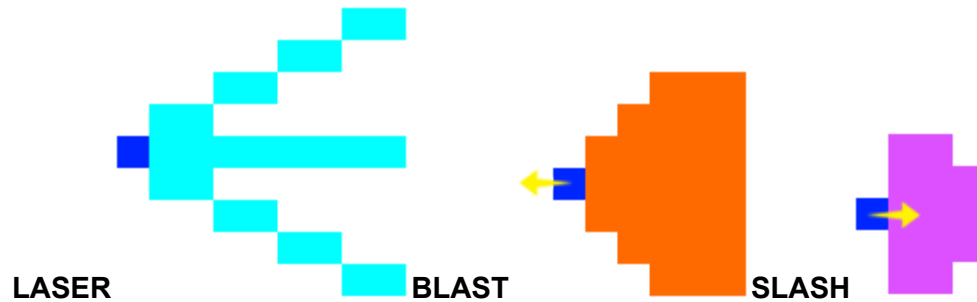
You lose if you touch an enemy.

You win if you survive for 60 seconds without touching an enemy.

Movement: Use the 4 movement buttons to move your avatar (the **blue pixel**) up, down, left and right, one tile at a time. You are allowed to move as fast as you can press the buttons.

Attacking: Use the 4 attack buttons to attack in the chosen direction. You are allowed to attack as fast as you can press the buttons, but every attack consumes 1 ammo and you cannot attack when you have 0 ammo.

Based on the chosen weapon, attacking creates a specific shape that eliminates overlapping enemies. The color of the attack is purely cosmetic.



In addition, BLAST and SLASH have special movement mechanics: the BLAST attack pushes you back one tile opposite of the attack direction, while the SLASH attack pushes you forward one tile in the attack direction.

Ammo management: The player has a maximum capacity of 12 ammo and starts fully stocked. To replenish ammo, you must collect the ammo pickup (**green pixel**) by moving onto it. Every ammo pickup grants 4 ammo. You cannot hold more than 12 ammo (ammo that goes over the limit is lost). The ammo pickup respawns immediately in a random location when picked up. Ammo will only spawn in the inner 12x12 square of the map (i.e. always at least 2 tiles away from the edge)

To warn the player of their ammo count in the midst of intense gameplay, the player's avatar (blue pixel) slowly turns white as they approach 0 ammo.

Enemies: The game starts with zero enemies (**red pixels**) on the map. Once the game begins, enemies spawn at a constant rate at a random tile on the edge of the board. Enemies will not spawn on existing enemies or the player. Once spawned, enemies will move 1 tile towards the player at a constant rate. Enemies can move diagonally.

Two or more enemies can move into the same tile, in which case they merge into a single enemy.

Points: For every enemy eliminated, the player gets one point.

Lose condition: The game ends if the player occupies the same tile as an enemy, which may occur when:

- The player moves into a tile occupied by an enemy.
- An enemy moves into the tile occupied by the player

Win condition: The player wins if they survive for 60 seconds without touching (occupying the same tile as) an enemy.

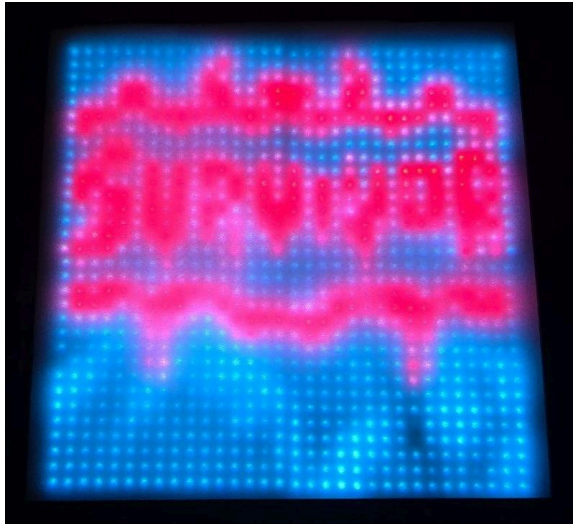
UI & UX

<Illustrate your prototype here, sketch or CAD drawing. Replace with photos of actual prototype when available>

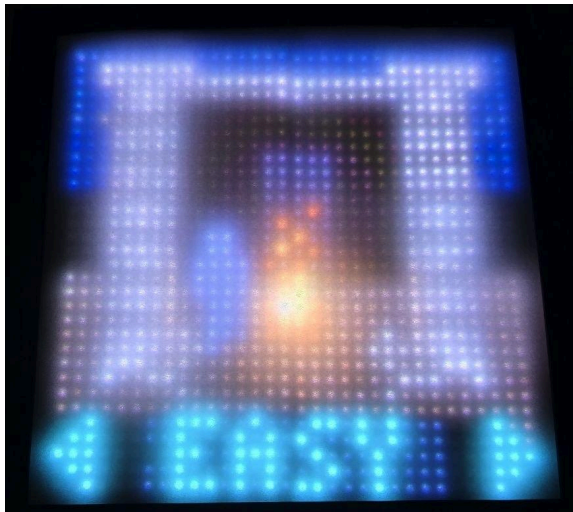
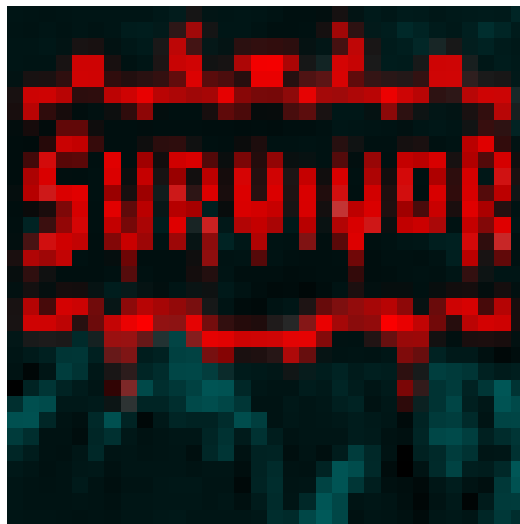
you have improved on it>

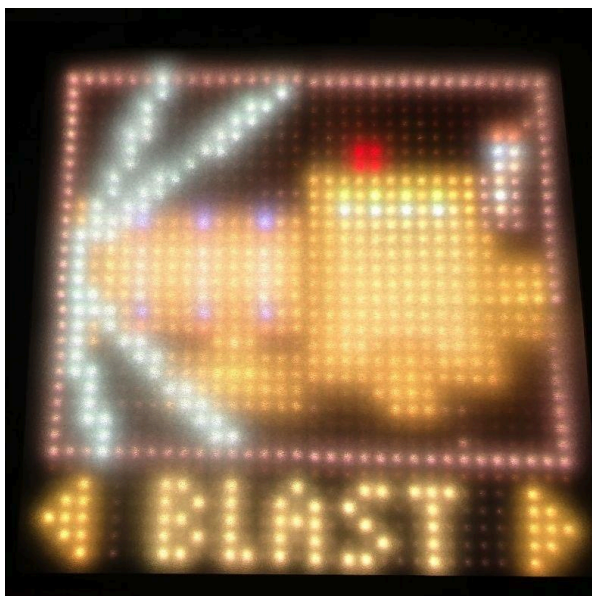
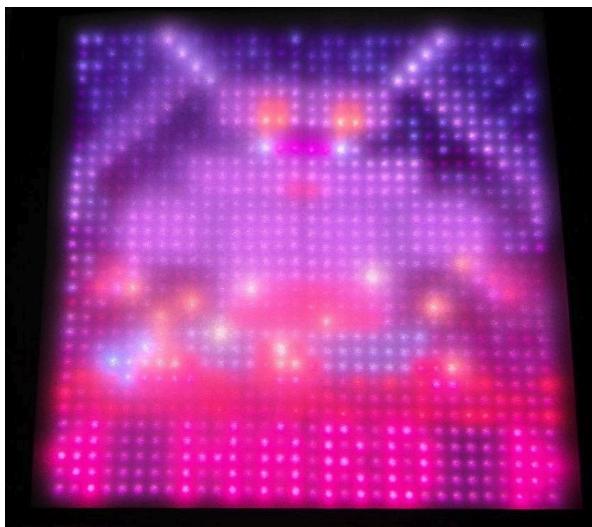
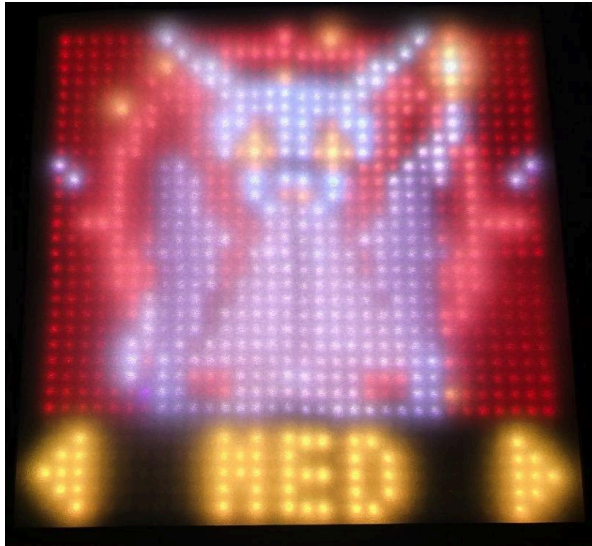


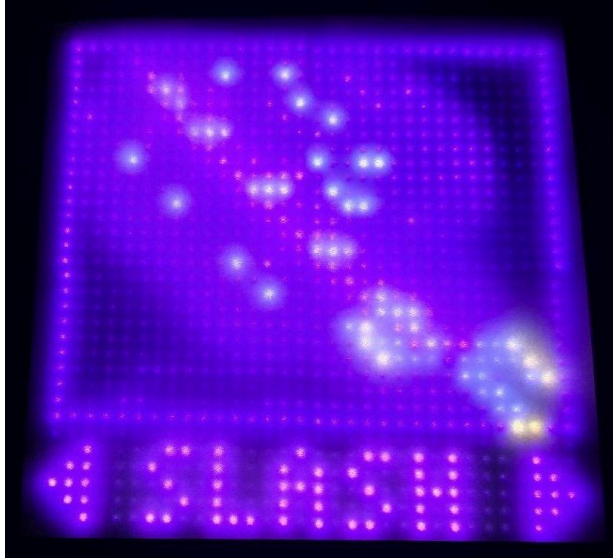
Actual screens:



Pixel Art:





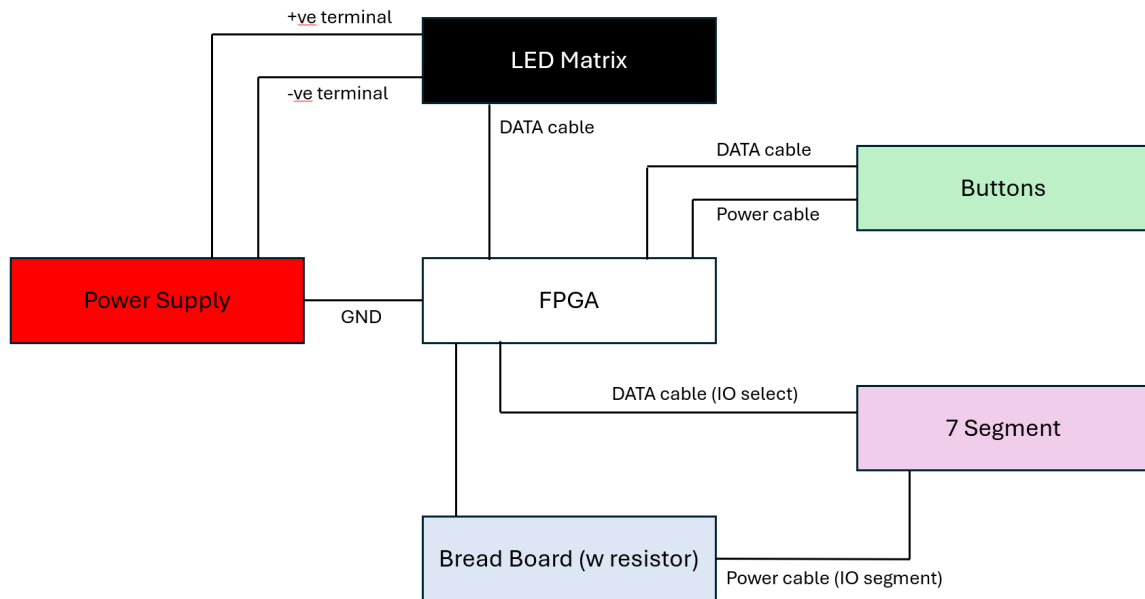




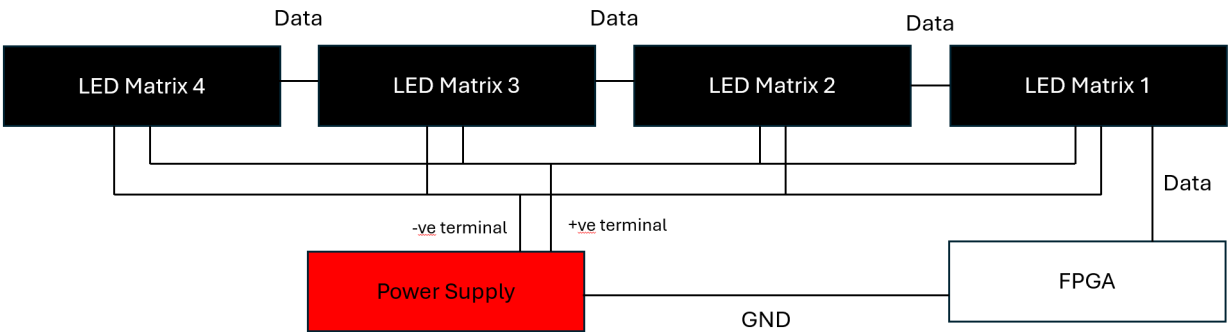
Electronic Design

Lots of diagrams are expected here.

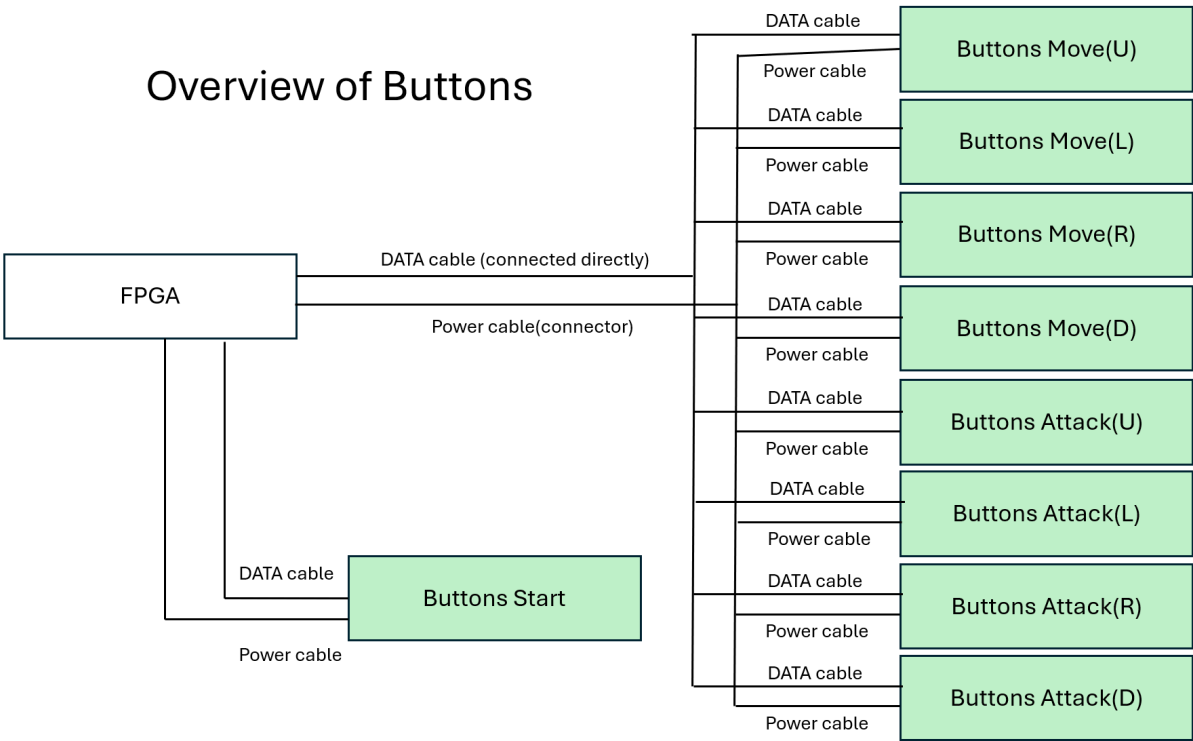
Overview of all wires



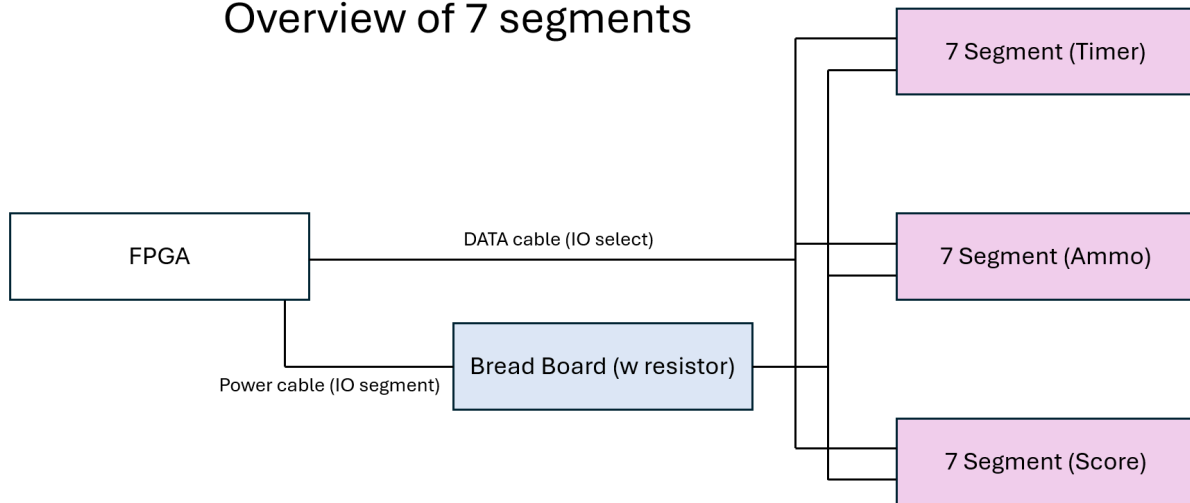
Overview of LED Matrix



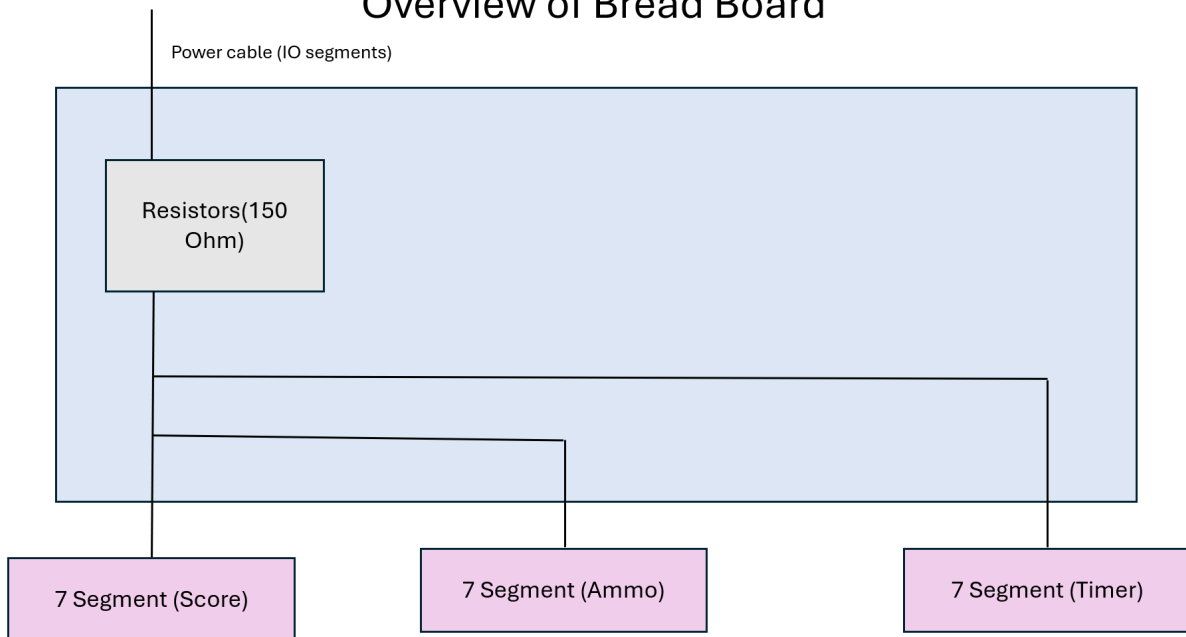
Overview of Buttons



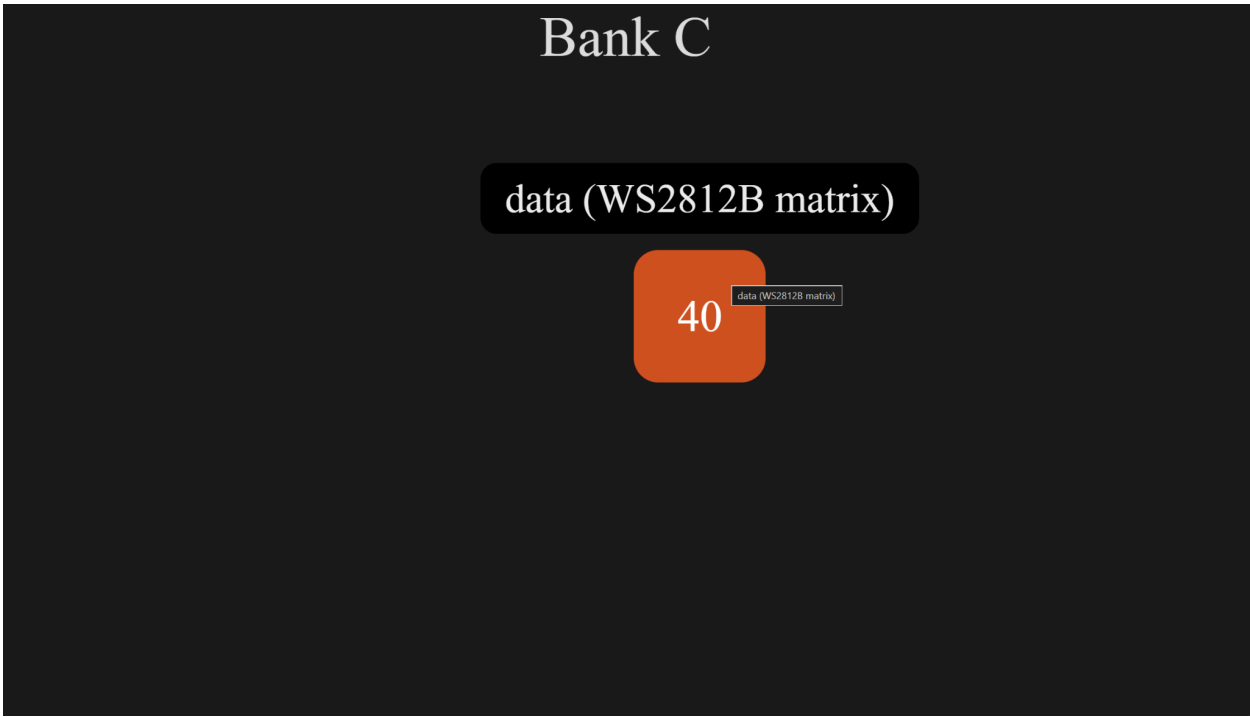
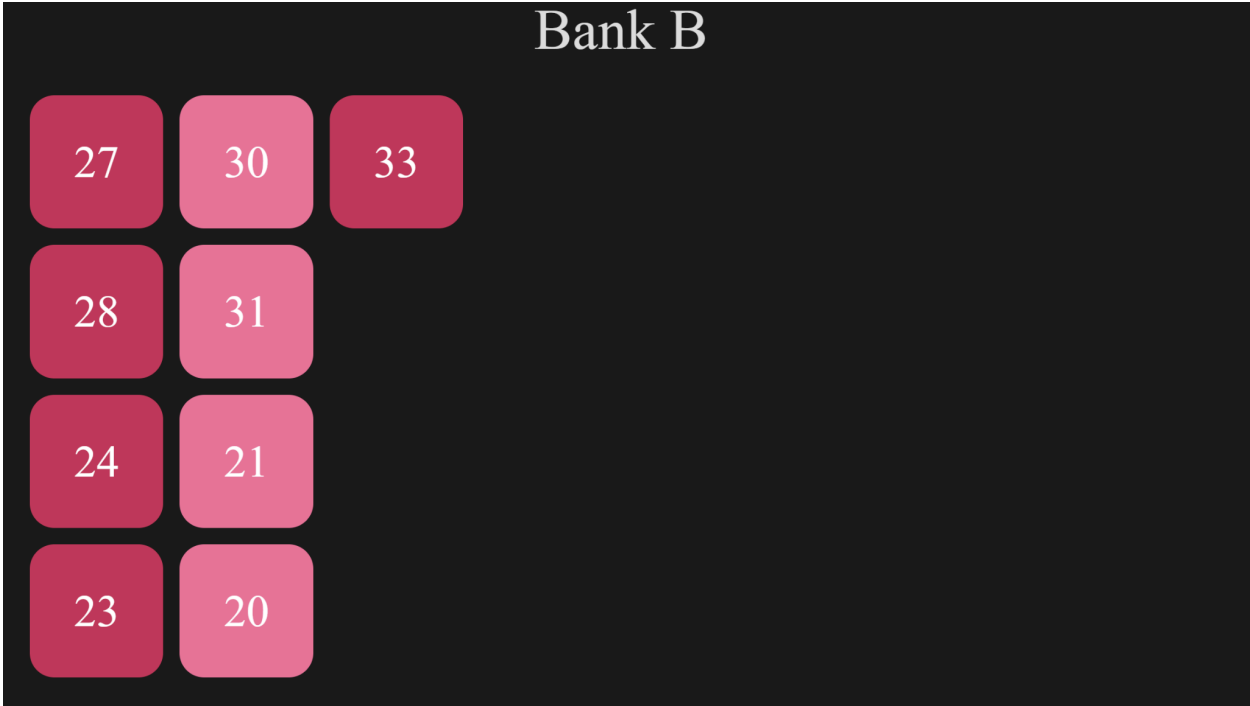
Overview of 7 segments



Overview of Bread Board







- Movement + start Attack (new)
- io_segment[0-6] (shared, no dot) io_select 4-dig (enemies)
- io_select2 4-dig (new) io_select3 2-dig (bullets)
- WS2812B data

Inputs: B28→ B27↑ B23↑ B24↓ (move) · B33 (start) · B20↑
 B21↓ B30→ B31← (attack)

Outputs: A5, A6, A48, A46, A45, A3, A2 (seg[0-6]) · A9, A8,
 A42, A43 (sel 4-dig enemies) · A11, A12, A40, A39 (sel2 4-dig) ·
 A14, A15, A37, A36 (sel3 2-dig) · C40 (matrix)

Input/Output

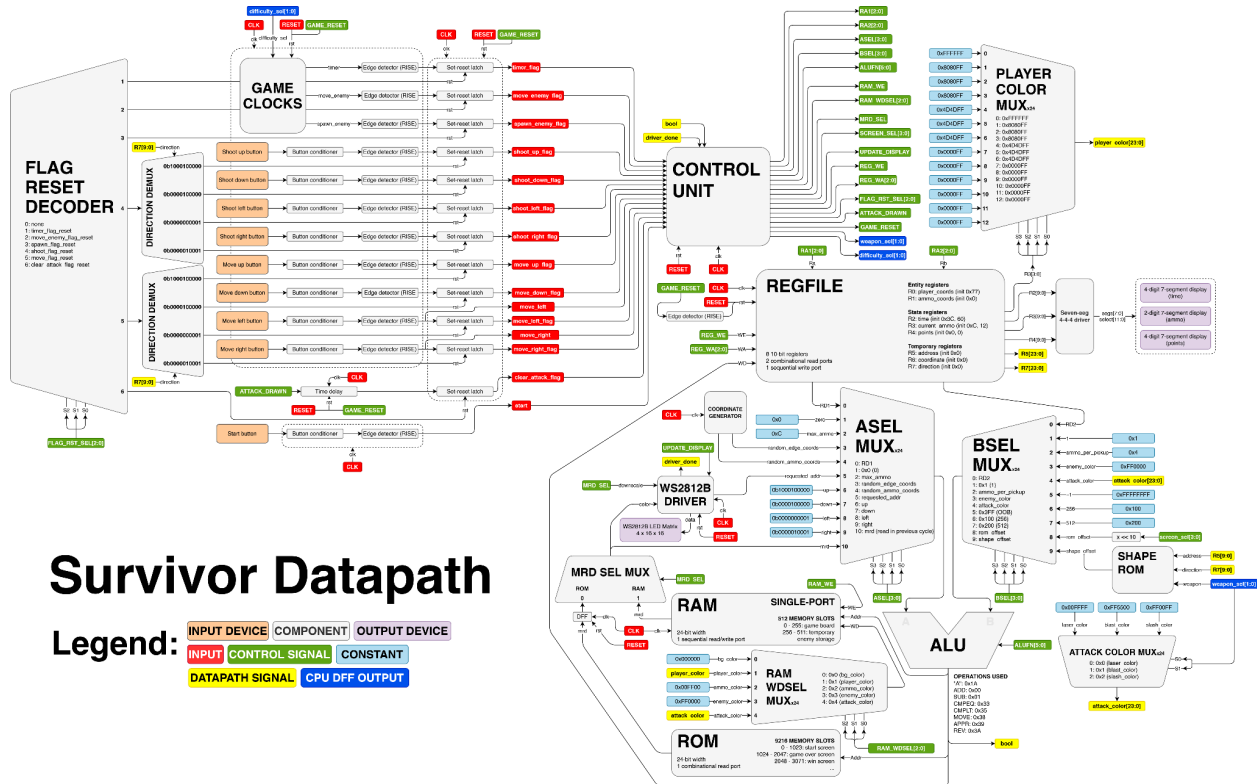
<Input and output devices for your final prototype, **all** expected values of output and input>

Type	Device	Values
Input	Buttons	'0' when not pressed, '1' when pressed
Output	WS2812B LED Matrix	Each LED receives a 24-bit RGB color value, where each R, G, B ranges from 0 to 255: 0x0 would be black/off and 0xFFFFFF would be white
	7-segment LEDs	Each segment is ON/OFF and overall displays digits 0-9

Datapath

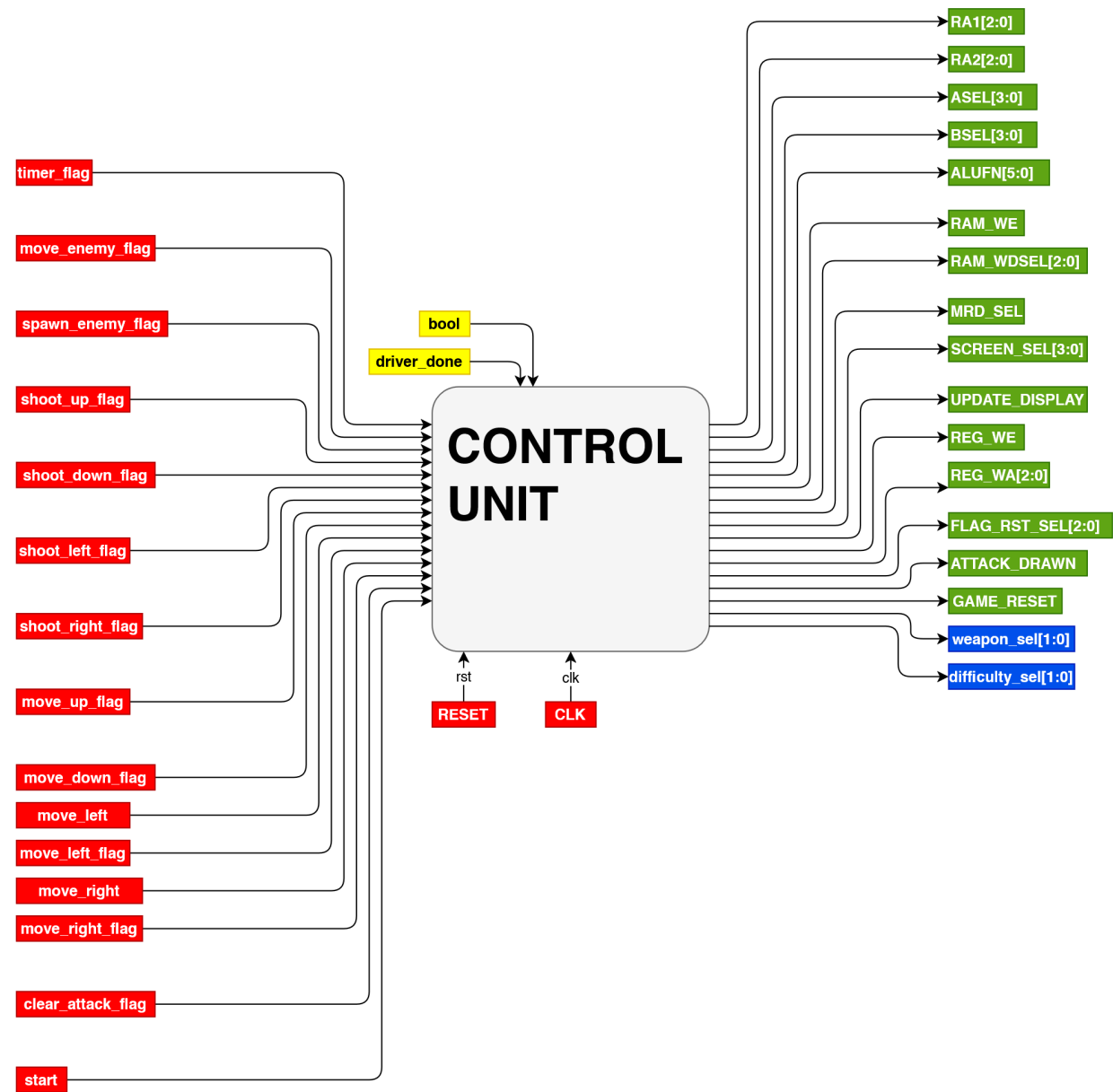
<Describe the datapath of your game, how many regs used, addressing, control signals, etc>

High-res download: [Survivor Datapath Final.png](#)



The entire datapath consists of 3 main modules: The **input controller**, the **control unit** and the main **datapath**.

Control Unit



The control unit receives inputs from the input controller and the datapath and generates control signals. Internally, it consists of:

- 3 registers for **state**, **weapon selected** and **difficulty selected**,
- FSM logic to control state transitions, and
- A ROM containing the control signals to output for each state.

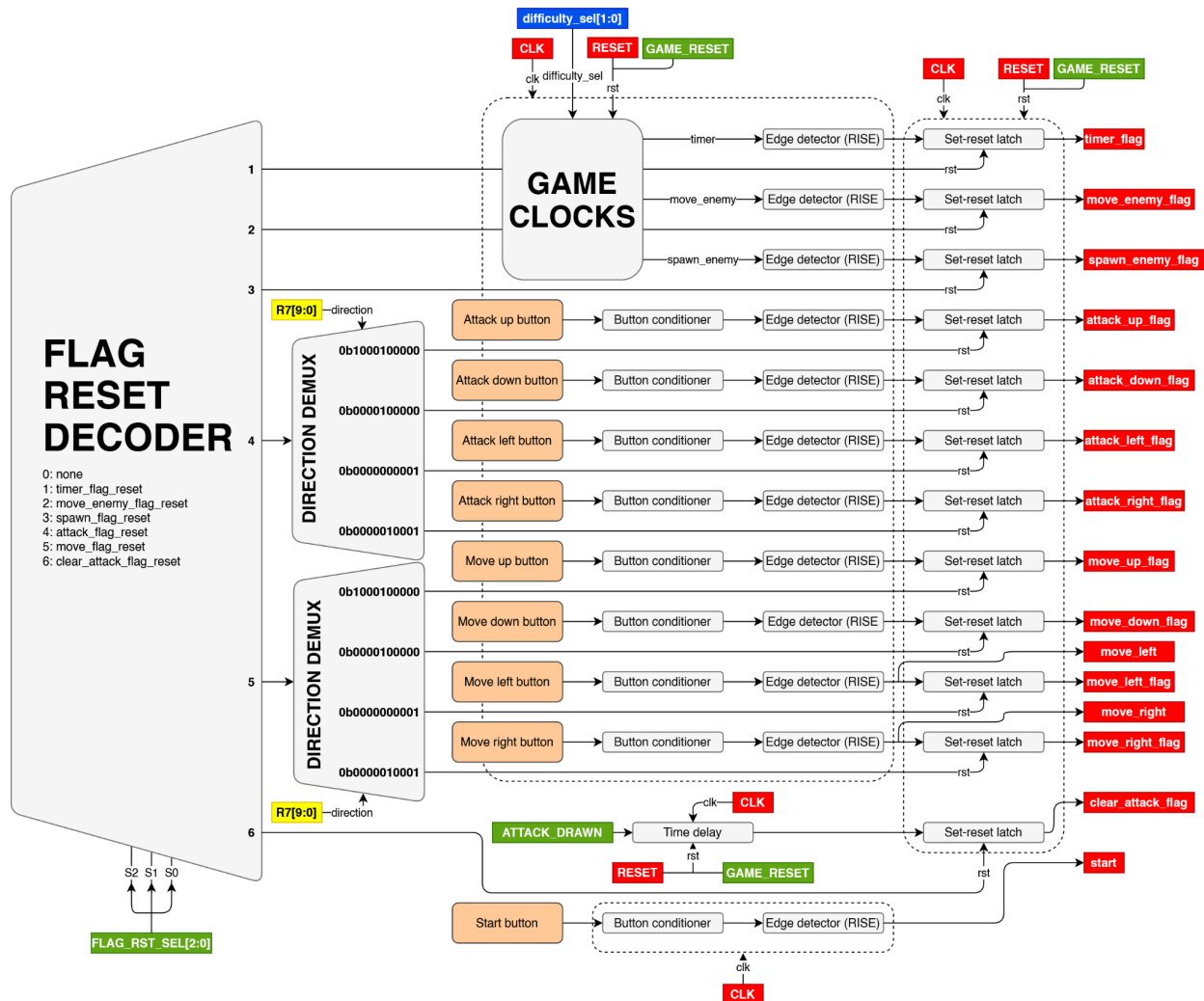
Control signals

Control signal	Destination	Purpose
----------------	-------------	---------

RA1[2:0]	REGFILE unit	Determines register data read (RD1) that goes to ASEL mux
RA2[2:0]	REGFILE unit	Determines register data read (RD2) that goes to BSEL mux
ASEL[3:0]	ASEL mux	Determines input A into the ALU
BSEL[3:0]	BSEL mux	Determines input B into the ALU
ALUFN[5:0]	ALU	Determines the operation performed by the ALU, including custom coordinate operations.
RAM_WE	RAM unit	Allows writing to RAM
RAM_WDSEL[2:0]	RAM unit	Determines the data (color) written into RAM
MRD_SEL	MRD SEL mux	Determines whether RAM or ROM data that was read in the previous cycle goes to the driver / ASEL mux. Data goes to the driver to display on the WS2812B LED matrices. Data goes to ASEL mux so that its color can be checked to tell what entity is at that location.
SCREEN_SEL[3:0]	BSEL mux	Is left-shifted by 10 to create the address offset for the WS2812B driver reading the selected screen data from the ROM. E.g. if SCREEN_SEL = 2 (win screen), $0x2 \ll 10 = 2048$, which points to the first address of the win screen image data. The offset is added to the address requested by the WS2812B driver to get the data for the desired screen.
UPDATE_DISPLAY	WS2812B driver	Triggers the WS2812B driver to start requesting addresses and sending color data to the WS2812B LED matrices
REG_WE	REGFILE unit	Allows writing to registers. The write data is always the ALU output.
REG_WA[2:0]	REGFILE unit	Determines the address to write to
FLAG_RST_SEL	Input controller's	Used to reset the latch for an event flag after the

	flag reset decoder	event has been handled by the FSM. When FLAG_RST_SEL = 0, no flag is reset.
ATTACK_DRAWN	Input controller	Sent once the pixels of an attack's shape are drawn. It goes to a time delay unit in the input controller, and the split-second delayed signal is then latched as clear_attack_flag, informing the FSM to clear the attack pixels.
GAME_RESET	Input controller and REGFILE unit	This signal is HIGH while the FSM is in its SETUP states. At the datapath, the rising edge of this signal triggers a REGFILE unit reset, wiping the values from the previous game. Only the rising edge is used because during setup for the next game, new ammo coordinates have to be generated and stored in a register before the game starts. At the input controller, it is used to reset and suppress (by constantly resetting) the clocks and latches, only releasing them the moment the game starts.
weapon_sel[1:0]	Shape ROM and Attack color mux	Determines the shape offset data and attack color used when drawing attacks. Every weapon has a different attack shape and color.
difficulty_sel[1:0]	Game clocks unit	Determines the frequencies of pulses for moving and spawning enemies which differs based on difficulty.

Input controller



The input controller is responsible for latching external player inputs and timer pulses into input signals for the control unit's FSM state transitions.

Outputs

Every set-reset latch outputs a flag representing a specific **event** in the game for the FSM to handle:

Output signal	Event
timer_flag	Decrement time
move_enemy_flag	Move enemies

spawn_enemy_flag	Spawn enemy
attack_up_flag	Attack up
attack_down_flag	Attack down
attack_left_flag	Attack left
attack_right_flag	Attack right
move_up_flag	Move player up
move_down_flag	Move player down
move_left_flag	Move player left
move_right_flag	Move player right
clear_attack_flag	Clear attack pixels

For detailed explanation on how these events are handled, see

[2026 50.002 Project Group Report Team 14](#)

The other outputs are used for menu navigation in the FSM:

move_left	Go to previous difficulty / weapon during difficulty/weapon selection screen
move_right	Go to next difficulty / weapon on difficulty/weapon selection screen
start	Progress to next screen (game over / win → start → difficulty selection → weapon selection → start game)

Components

Game clocks: Contains variable pulser units: 0.5hz, 1hz and 3hz. The timer output (which decrements the time) is always 1hz. Based on difficulty_sel, different frequencies for moving and spawning enemies are output for each difficulty.

Set-reset latch: Outputs low until it receives HIGH, then outputs HIGH until reset.

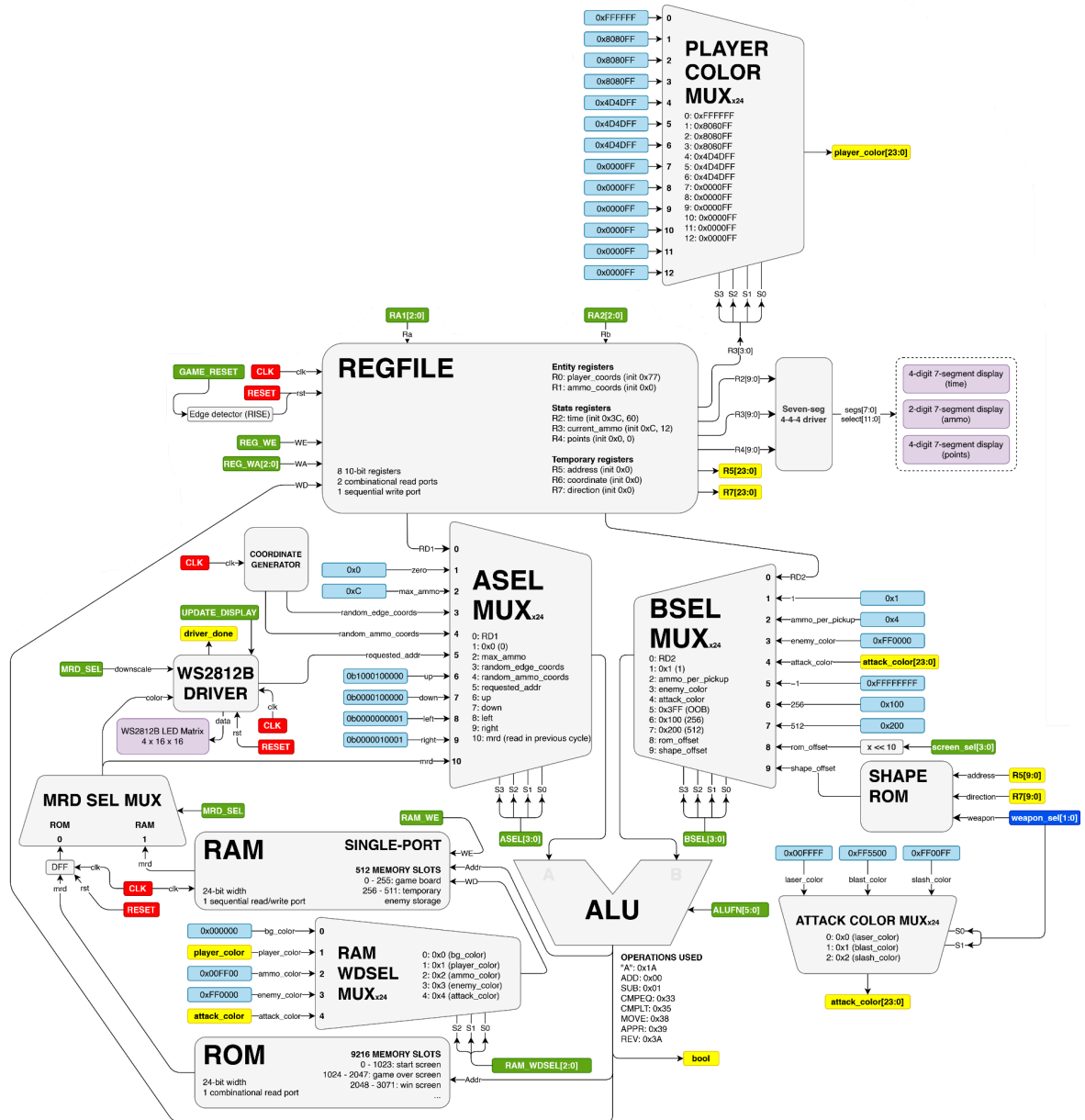
Flag reset decoder: Based on the FLAG_RST_SEL control signal, an output line is selected and used to reset an event flag. A decoder was used instead of a mux to remove the need for a control signal: no reset takes place when 0 is selected.

Direction demux: Based on a 10-bit encoded direction, it routes the move / attack flag reset signal to the correct latch.

Time delay: When it detects a rising edge, it outputs a 1-cycle HIGH a split second later. If it detects another rising edge while it is currently delaying a signal, it resets the delay.

Main Datapath

The main datapath is highly inspired by the Beta CPU, modified for the framework of our game.



For explanation on coordinates and direction encoding used throughout the datapath, see

2026 50.002 Project Group Report Team 14

Components

Component	Specifications / inputs	Explanation
REGFILE unit	8 10-bit registers 2 combinational read ports 1 sequential write port Entity registers R0: player_coords (init 0x77) R1: ammo_coords (init 0x0) Stats registers R2: time (init 0x3C, 60) R3: current_ammo (init 0xC, 12) R4: points (init 0x0, 0) Temporary registers R5: address (init 0x0) R6: coordinate (init 0x0) R7: direction (init 0x0)	R0 and R1 store the coordinates of the player and ammo pickup on the map, so that the entire map does not need to be scanned in order to locate them. R2, R3 and R4 store game stats. These are exposed to the seven-seg driver for external display. R5, R6 and R7 are “temporary” registers. These registers are used by the FSM to store values required to accomplish a task. R5 (address) is used to store a value that is continually incremented in order to “scan” the RAM R6 (coordinate) stores an 8-bit encoded coordinate (e.g. an enemy’s next position) so that it can be checked for various conditions (e.g. overlap with player) for game logic before finally being used as an address to draw the entity. R7 (direction) stores a 10-bit encoded direction (up, down, left or right), which is used for moving the player and clearing the correct flag after the player moves or shoots.
Seven-seg 4-4-4 driver	-	Reads the time (R2), current ammo (R3) and points (R4) registers in the REGFILE unit and converts the values into segments and select bits to output these values across the three 7-segment displays.
Coordinate generator	-	Generates a “random” edge coordinate and ammo zone coordinate. There are 60 edge coordinates along the edge of the 16x16 board. There are 144 ammo zone coordinates in the center 12x12 square of the board. The generator is not pseudorandom; it contains a counter that is used to select a

		coordinate from a scrambled ROM.
ASEL mux	0: RD1 1: 0x0 (0) 2: max_ammo 3: random_edge_coords 4: random_ammo_coords 5: requested_addr 6: up 7: down 8: left 9: right 10: mrd (read in previous cycle)	0 is used to get a register value. 1 is used as a value to write to R5, resetting the address for a loop. 2 is used to check if the player has exceeded the maximum ammo count. 3 and 4 are used when spawning enemies or ammo pickups, saved to R6 first for various checks in the FSM. 5 is used during an update of the LED display. As long as the WS2812B driver is active, this is selected, forming a loop in the datapath that allows the driver to display RAM/ROM data. 6, 7, 8 and 9 are used to write a direction into R7 when the player moves or shoots, which is then read by the direction muxes and shape ROM. 10 is used to immediately perform an operation on memory data read in the previous cycle (since both ROM and RAM reads are delayed by one clock cycle), forgoing the need to write to a register first.
BSEL mux	0: RD2 1: 0x1 (1) 2: ammo_per_pickup 3: enemy_color 4: attack_color 5: 0x3FF (OOB) 6: 0x100 (256) 7: 0x200 (512) 8: rom_offset 9: shape_offset	0 is used to get a register value. 1 is used to increment R5 or check if the player is out of ammo (if ammo < 1). 2 is used to increase the player's ammo count when they pick up ammo. 3 is used to check if a pixel read from RAM represents an enemy. 4 is used to check if a pixel read from RAM represents an attack pixel. 5 is used to check if a computed coordinate is out of bounds. 6 and 7 are used to check whether a loop is complete (once all 256 tiles of the board have been scanned, or once all 512 memory slots have been reset). 256 is also used to offset the R5 address to access temporary enemy storage during the enemy move event. 8 is used to offset the address requested by the WS2812B driver to the correct screen. 9 is used to add to the player's coordinates to get the coordinates of a pixel in the attack

		shape in order to draw it.
Player color mux	0: 0xFFFFFF 1: 0x8080FF 2: 0x8080FF 3: 0x8080FF 4: 0x4D4DFF 5: 0x4D4DFF 6: 0x4D4DFF 7: 0x0000FF 8: 0x0000FF 9: 0x0000FF 10: 0x0000FF 11: 0x0000FF 12: 0x0000FF	Based on the player's ammo count, read from the register (R3), outputs a 24-bit RGB value between blue and white. This value goes to the RAM WDSEL mux and is used to redraw the player when their ammo count changes.
Shape ROM	552 slots 10-bit width	Contains sets of 10-bit offsets used to draw attack shapes. When an attack shape is being drawn, based on the address, direction and weapon selected, a 10-bit offset is provided. These offsets are added to the player's coordinates to get the coordinate of an attack pixel. The complete shape is drawn by incrementing the address to get every coordinate in the shape. In the ROM, the end of an attack shape is indicated by an offset of (0, 0) (10b0).
Attack color mux	24-bit width	Based on the selected weapon, a different 24-bit RGB value is output to the WDSEL mux and used to draw attack pixels.
ALU	32-bit operations	Performs all arithmetic operations. The least significant bit of the ALU output ("bool") is sent directly to the control unit. This allows the control unit to branch immediately after a compare operation. The ALU output also goes to the REGFILE unit as write data.
RAM	512 memory slots	The RAM stores 24-bit RGB color data

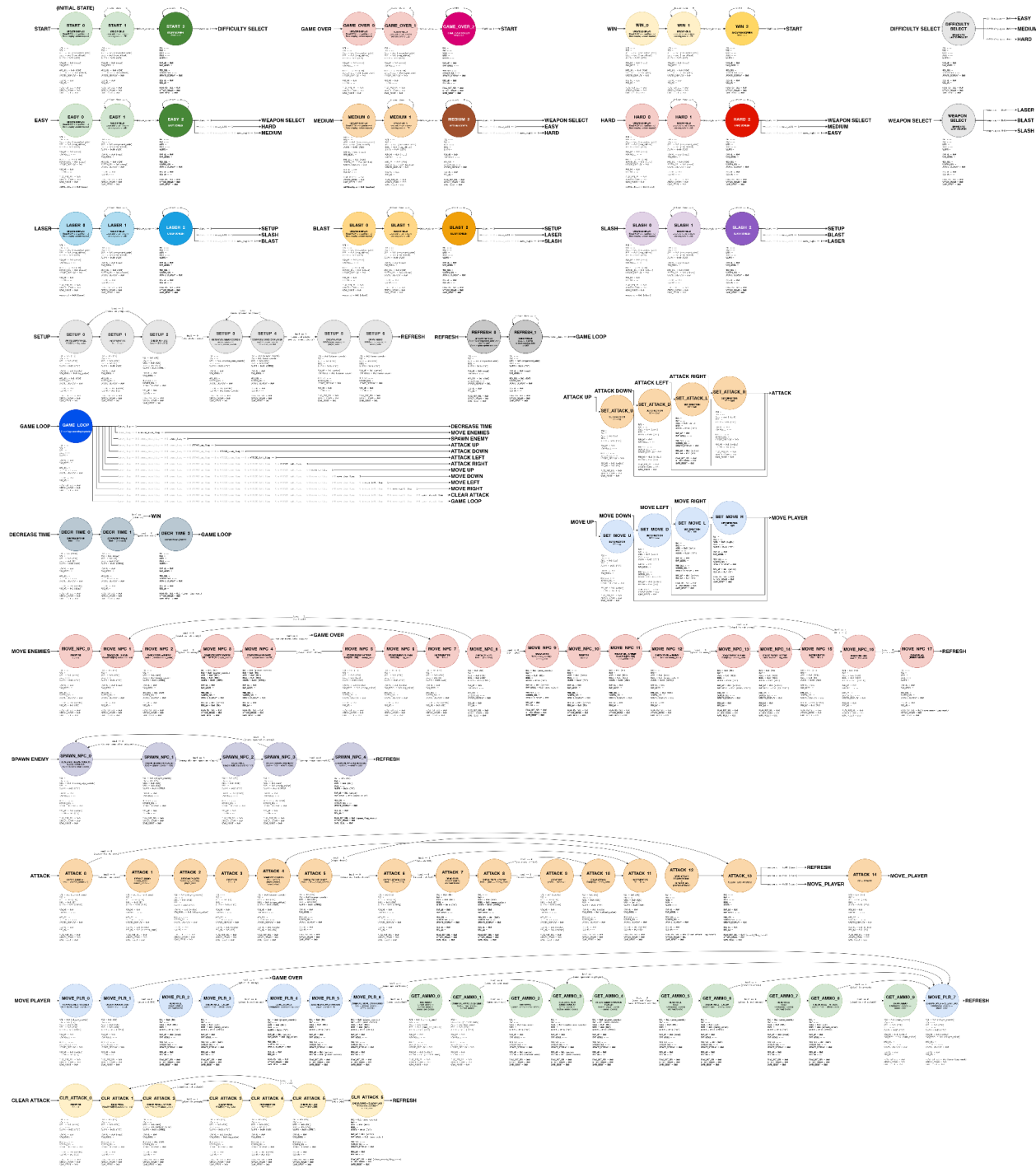
	24-bit width 1 sequential read/write port	representing the entire game board and all the entities on it in slots 0 - 255. As the game takes place, this data is constantly overwritten and re-displayed on the external LED matrix. Slots 256-511 are used as temporary enemy storage, which is used only during the enemy move event.
RAM WDSEL mux	0: 0x0 (bg_color) 1: 0x1 (player_color) 2: 0x2 (ammo_color) 3: 0x3 (enemy_color) 4: 0x4 (attack_color)	Outputs a color to be used as write data to the RAM. Player color and attack color come from the player color mux and attack color mux, and varies based on the current ammo and selected weapon.
ROM	9216 memory slots 24-bit width 1 combinational read port	Contains the image data (24-bit RGB values) for the 9 screens in the game. Every screen takes up 1024 memory slots. Slot: 0 - 1023: start screen 1024 - 2047: game over screen 2048 - 3071: win screen 3072 - 4095: easy screen 4096 - 5119: medium screen 5120 - 6143: hard screen 6144 - 7167: laser screen 7168 - 8191: blast screen 8192 - 9215: slash screen
MRD SEL mux	0: ROM mrd 1: RAM mrd	Controls whether RAM or ROM data is sent to the WS2812B driver for display. The ROM data passes through a DFF before it reaches the MRD SEL mux to delay the memory data by one clock cycle (similar to RAM). This allows the WS2812B driver to behave the same way when reading from either RAM and ROM is required.
WS2812B driver	-	Takes in the 24-bit RGB values from the RAM / ROM and outputs pulse-encoded signals to the WS2812B led matrices to display the image.

		The driver was modified to take RGB values instead of GRB. It also contains an internal index mapper and downscaler to correctly output to the four matrices in series. The mapper is used to correctly map addresses to LED index across the 4 serpentine LED matrices. The downscaler is used to “downscale” an address from 0-1023 (32x32) to 0-255 (16x16). This allows the 16x16 game board to be displayed on the 32x32 LED, with every pixel being drawn as 4 pixels. MRD_SEL is used to activate the downscaling, because when MRD_SEL is 1, 16x16 RAM data is being read.
--	--	---

FSM

<Describe your game state machine and all control signal values + output at each state>

High-res download:  Survivor FSM Final.png



How to read the diagram

All of the linear states involved in accomplishing a specific task are named, numbered and coloured as a group.

Each group of states starts and ends with a **label**. Rather than draw arrows between every single state transition, which would be extremely messy, we use labels to group states together and simplify large jumps across the diagram.

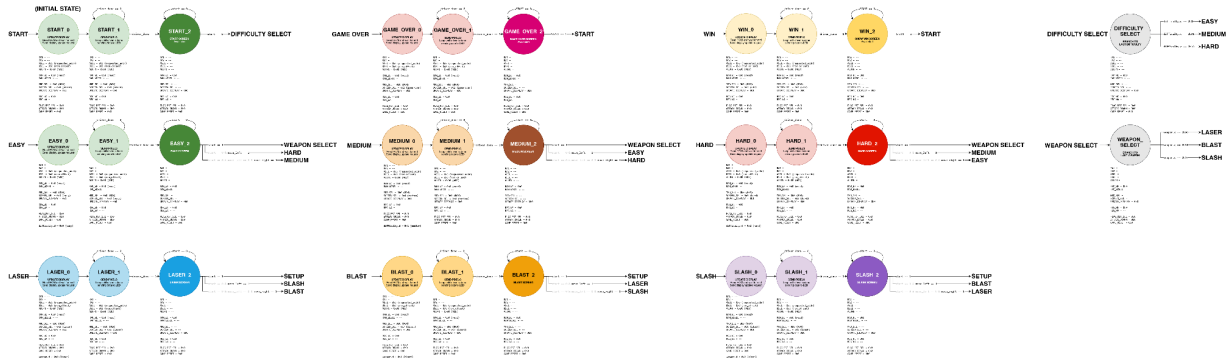
To make the FSM easier to understand, each state circle contains an explanation of what the control signals for that state accomplish in the datapath, in the form of a textual description or pseudocode.

All control signals are written below the state circle.

Broad explanation

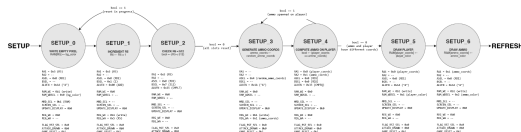
(Refer to the high-res image - the images below are just to indicate what states are being referred to)

Menu states



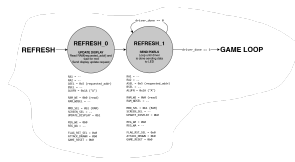
These states handle menu navigation. Every time a new screen is displayed, the screen data from the ROM is read and the WS2812B driver is engaged to display the image.

Setup states



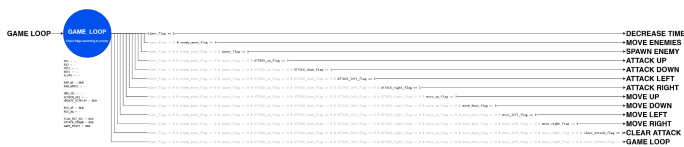
These states set up the next game by wiping the RAM (game board and temporary enemy storage), generating and storing the initial ammo pickup coordinates, and drawing the ammo pickup and player on the game board (RAM). It then transitions to REFRESH to display the game board and start the game.

Refresh states



These states refresh the screen during the game. The game board in the RAM is read and the WS2812B driver is engaged to display the game board, then transitions to game loop.

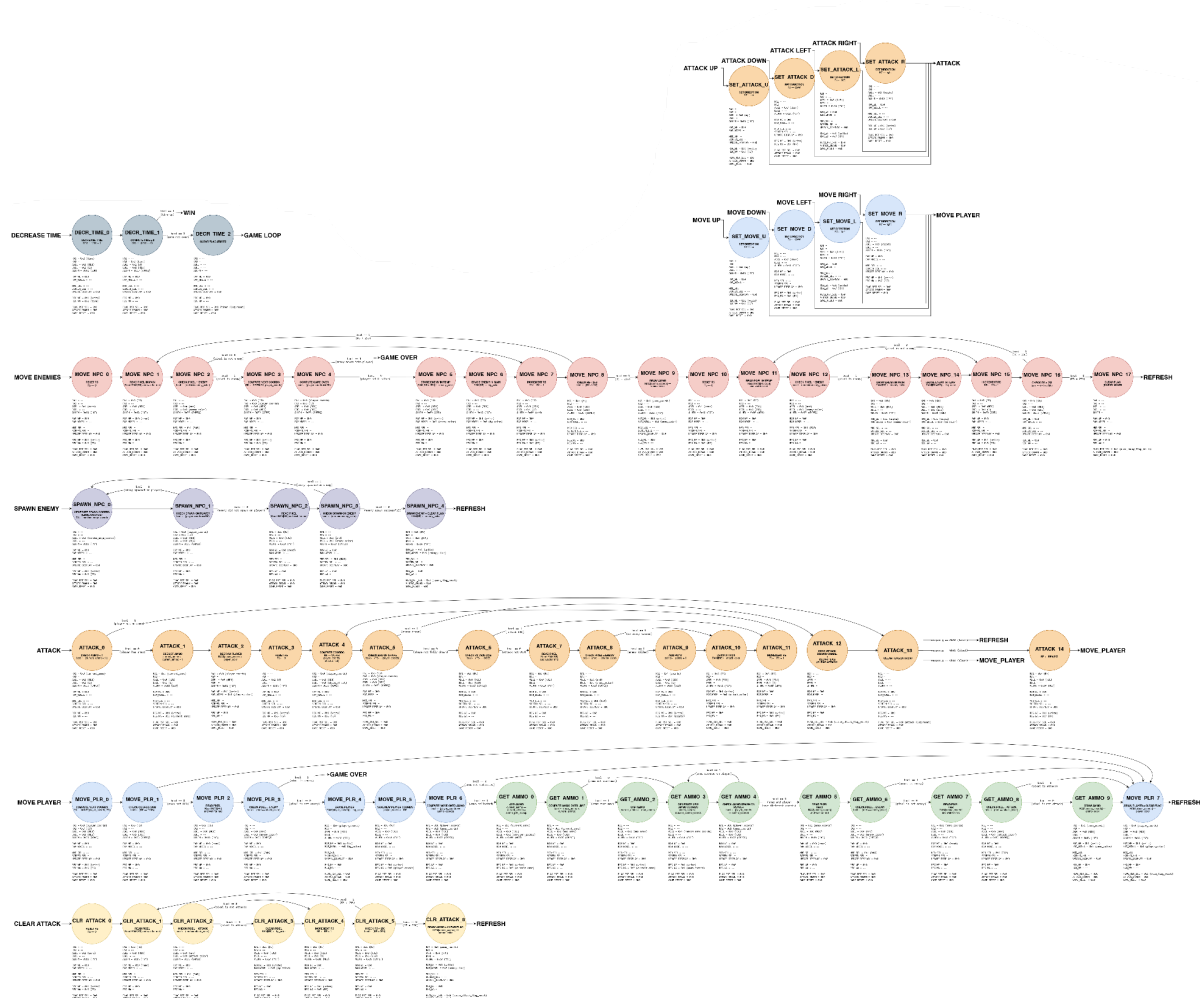
Game loop



The FSM will be at this state when nothing is happening while the game is running. All of the event flags from the input controller are checked in order of priority. If a flag is high, the FSM transitions to the series of states designed to handle that event.

The priority matters - for example, decreasing time needs to take precedence over moving enemies so the player does not lose at the last second, and moving enemies needs to take precedence over spawning enemies to prevent enemies from moving immediately after being spawned.

Events



Each color-coded series of events handles a specific game event, then transitions to the refresh states.

Event	How the event is handled
DECREASE TIME	Register R2 (which stores remaining time) has its value decreased by 1. If R2 is 0, 60 seconds have passed and the FSM transitions to the WIN screen states.
MOVE ENEMIES	R5 is reset to 0. R5 serves as the RAM address to scan the game board. Every pixel on the game board (RAM slots 0-255) is read. Its color is compared to enemy_color. If the pixel is an enemy, its next position as it approaches the player is computed and stored in R6. If this next position is the player's position, FSM transitions to the game over screen states. Otherwise, the enemy in its next position is drawn in temporary enemy storage (RAM slots 256-511) and the pixel on the game board is cleared.

	After all the enemies on the game board have been cleared, the ammo pickup is redrawn (because enemies are drawn over the ammo pickup, and moving an enemy may reveal the ammo pickup). Then, the enemies in temporary enemy storage are copied back onto the game board and cleared. Temporary enemy storage is necessary because if an enemy was moved directly on the game board, it could be scanned and moved again since the board is scanned linearly.
SPAWN ENEMY	An enemy is drawn at a random edge coordinate from the coordinate generator, by writing the 24-bit color “red” representing an enemy into the corresponding address in the RAM game board.
ATTACK UP	First, the direction of the attack is stored in R7, which is accessed by the shape ROM to get the correct attack shape, and later by the input controller’s direction demuxes to reset the correct latch. The player’s ammo is checked. If they have ammo, it is deducted and the player is redrawn (since the player colour changes with ammo count). R5 is reset to 0. The R5 value serves as the ROM address for getting the offsets of the current weapon’s attack shape. The offset from the shape ROM is added with the player’s position to get the coordinate of an attack pixel, stored in R6. In the shape ROM, the end of a shape is indicated with an offset of (0, 0), hence if the coordinate computed is the same as the player’s position, it indicates that the shape is over. Otherwise, if it is not out of bounds, the pixel on the game board is checked. If it is an enemy, a point is added. The attack pixel is drawn at the computed coordinate. Once the shape has been fully drawn, the ATTACK_DRAWN signal is sent to the input controller, which triggers clearing the attack pixels later. The clear attack signal is also cleared to prevent the attack from being cleared immediately after it is drawn. Finally, the attack flag itself is cleared.
ATTACK DOWN	
ATTACK LEFT	
ATTACK RIGHT	
MOVE UP	First, the direction of the player move is stored in R7, which is accessed by the input controller’s direction demuxes to reset the correct latch. The player’s next coordinate is computed and stored in R6. If it is out of bounds, nothing happens and the flag is cleared. Otherwise, the pixel at this coordinate is read. If it is an enemy, transition to the game over screen states. Otherwise, erase the player pixel and redraw it at the new location, and update the player position register R0. If the new position is
MOVE DOWN	

	the position of the ammo pickup, transition to the get ammo states. Otherwise, draw the player and clear the move player flag.
MOVE LEFT	
MOVE RIGHT	
GET AMMO	Since the player can only get ammo by moving, this event is contained within the move player event. The player's ammo (register R3) is increased. If this new value is greater than the maximum of 12, it is capped at 12. New ammo coords from the coordinate generator are stored in R1 (ammo coords). If these coords are the same as the player's (ammo cannot spawn on the player), a new coordinate is stored. The pixel at this coordinate is then checked - if it is not an enemy or attack pixel (enemies and attack pixels take precedence), the ammo pickup is drawn in RAM.
CLEAR ATTACK	R5 is reset to 0. The R5 value serves as the RAM address to scan the game board. Every pixel on the game board is read. Its colour is compared to attack_color. If it is an attack pixel, it is cleared by replacement with the background color. Finally, the ammo pixel is redrawn (because attack pixels are drawn over the ammo pickup, and clearing a laser may reveal the ammo pickup) and the flag is cleared. The reason why clearing the attack pixels is a separate event (and not combined with the attack event) is so that the pixels are only cleared after they have been visible for a brief moment, while not freezing the game until they need to be cleared.

Budget

<Components and their cost>

8 March	Item	Qty	Cost/item	#	Total
1	Buttons	11	0.66		7.26
2	Spring Terminal	2	0.43		0.86
3	Jumper Wires (MM)	2	0.67		1.34
4	Jumper Wires (MF)	2	0.65		1.3
5	Jumper Wires (FF)	2	0.7		1.4
6	8 segment LED	4	0.51		2.04
7	Power Adapter	1	1.68		1.68
8	Wood Glue	1	3.12		3.12
9	Red and Black Spray Paint	2	3.9		7.8
10	jumper wires (50cm) 40 pcs	1	5.86		5.86
11	jumper wires (50cm) 40 pcs	1	4.03		4.03
12	jumper wires (50cm) 40 pcs	1	4.34		4.34
13	7 segment LED (5 pcs)	2			4.67
11	jumper wires (50cm) 40 pcs	1	1.99		1.99
14	Red Acrylic Paint	1	4		4
15	Blue tack	1	3.14		3.14
16	Shipping	-	-		4
					58.83

2D

<Write about efforts you've put into your project that aligns with 2D>

Hardware Optimisation

Pipelining

Our FSM employs pipelining through its general-purpose CPU design and the use of 3 temporary registers (R5, R6 and R7). Instead of using specialized combinational logic units to attempt to perform complicated operations within one clock cycle, we **break complex tasks up across several clock cycles, fully utilizing our ALU** for all arithmetic operations (addition, subtraction, comparison, and custom coordinate operations).

Data Structure

RAM

In order to store our game data, we use a single-port RAM to store data of the entities on the board (the 24-bit RGB values can be sent to the WS2812B driver directly). We used a single-port RAM to simplify to reduce the number of control signals required (only one address vs. both read and write address) as there is no situation where we need to read and write to the RAM simultaneously.

Initially, we considered encoding the entities in fewer bits since there are only 5 (background, player, ammo, enemy, laser) as we thought this would reduce latency. However, upon further research, we learned that the read/write speed of block RAM on the FPGA is not affected by the width of the entries. Encoding the entities would require an additional 8-to-1 mux to convert the data back to colours for the WS2812B driver, which would in fact increase latency.

ROM

We use ROMs to store all unchanging data required in the game, such as screen images, attack shape offsets, and control signals for each state.

Addressing scheme

Our 16x16 coordinate system represents a coordinate in an 8-bit value by concatenating the 4-bit row index and column index. For example, 0xFF represents the coordinate (15, 15).

Since our game takes place on a 16x16 display (256 squares), we store both RAM and ROM data at contiguous memory addresses starting from address zero, in “frames” of 256 memory slots. This results in no memory space being wasted.

Initially, we used a 32-bit coordinate system which used the high 16 bits for the row index and the low 16-bits for the column index (e.g. (4, 2) would be 0x00040002). Since we only plan to

run the game on a 16x16 board, we realized that we only need 8 bits in total (4 for row index and 4 for column index). We also realized that by concatenating these bits directly, the resulting 8-bit coordinate system allows coordinates to be used directly as memory addresses. For example, coordinates (4, 2) encoded as 0x42 can be used directly as address 66, meaning no additional conversion logic is necessary. This allowed us to remove an ALU operation to convert the coordinate into an address, remove ROM data that was being used to look up coordinates from addresses, and remove states in our FSM for these conversions.

Contiguous storage allows the board to be easily scanned in a linear manner through incrementing the address, which is done by both the WS2812B driver and FSM.

This frame storage system allows us to store all the RAM/ROM data that needs to be read by the WS2812B driver in one RAM unit and one ROM unit, instead of having multiple and selecting different RAM/ROM units using a mux, saving resources and reducing control signals required.

Registers

Effort was made during the design of our FSM to minimize the number of required registers to 8 so that they are addressable by 3 bytes.

The register width is minimized to 10 bits, which is the minimum width for any value that may be stored in a register (specifically 10-bit encoded direction).

Enhanced Game Features, UI & UX Enhancement

After achieving our first working prototype, we made many significant improvements to the game mechanics, UI and UX.

Larger display

We enlarged the display from the single 16x16 LED matrix used for testing to the current 32x32 LED display made from 4 matrices. The game remains running efficiently on a 16x16 board in RAM, with each pixel upscaled to 4 pixels on the larger display, significantly improving visibility without sacrificing performance. The meticulously designed screen images are in full 32x32 resolution to make full use of the larger display and display stunning screen art for the player's enjoyment.

Difficulty system

To cater to players of different skill levels, we added different difficulties, which change the movement and spawn rates of enemies. During initial playtests, we found that many people were overwhelmed by the difficulty and lost too quickly to learn how to play the game. In addition to making the game more accessible, the incremental difficulties allow new players to learn the game slowly and take on harder difficulties as their skill improves, something we

observed firsthand with new playtesters. This created a more accessible, more rewarding experience for our relatively difficult game.

Weapons

Initially, the only attack available was a straight laser. We decided to implement 2 new weapons, SLASH and BLAST, to create gameplay variety and replay value. Each weapon has a different attack shape, catering to players with different play styles. In addition, BLAST and SLASH have special movement mechanics (BLAST pushes the player back while SLASH moves them forward), which are used to balance the weapons as well as make them more diverse and interesting. Each weapon's attacks appear as a different color, giving them a unique identity and memorability.

We found that the game was too ultimately too difficult with the original single straight laser, which felt weak and made high spawn rates on the higher difficulties feel unfair. We opted to make the new weapons strong and improved the laser to match by adding two additional lasers, making combat easier and more fun as more enemies can be eliminated and more points gained.

Ammo reminder

Ammo management is a core mechanic in our game. A common point of frustration was that during intense moments in the game, it was difficult to look at the ammo count on the 7-segment display, which often resulted in players running out of ammo, trying to attack unsuccessfully and then moving right into an enemy they thought would be eliminated, resulting in a game over. It is also difficult to attempt to keep count of ammo mentally to avoid this issue.

We addressed this issue by implementing a mechanic where the player's avatar (blue pixel) would slowly turn white as they approached zero ammo, warning them that they are about to run out and to seek the next ammo pickup immediately. This subtle change allowed players to manage their ammo much more easily without taking their eyes off the game during critical moments.

Sustainability and Inclusivity

<Write about sustainability and inclusivity aspect of your hardware prototype>

To allow for usability in the future, we reduced the usage of adhesive and soldering to the minimum. For the majority of the components, we used nails to hold everything together, allowing it to be easily disassembled and recycled. Additionally, for our circuitry, instead of soldering wires together, we use connectors and breadboards, so that all the jumper wires and electrical components used can be further used for other projects in the future. We only used adhesives for parts that are unable to be reused, such as the acrylic words on the wooden

surface. We have also taken note to use wood glue to allow the acrylic words to be removed from the wooden boards so they can be recycled in the future.

Additionally, even though we had the shopping list for our project decided on by week 5, we purposefully waited until the weekend of week 6 to buy our materials. The reason for us doing that would be to purposefully source and reuse the materials that have been recycled from the IDEa showcase on Week 6 Friday. As seen in our purchase history, a large portion of materials have not been bought, but actually found by us looking through the discarded components by the juniors, reducing waste in the environment.

For the inclusivity component, we have also implemented 3 different difficulties so that players with different skill levels and motor skills are able to play. Through rigorous playtesting, we have ensured that the easy difficulty is such that even an elderly, child or a person with motor disability is able to play and have a chance of winning the game.

Summary

<Conclusion, lessons learned, overall experience>

This project gave us a new perspective on designing programs - not just designing a game, but also taking into consideration the hardware it runs on, down to the most basic level, and designing a datapath to run that game most efficiently and elegantly.

An important lesson would be to precisely define the scope of the game beforehand. After we achieved our original goal, we decided to implement additional features and mechanics which often required changing the datapath drastically. Every time we changed the datapath, new bugs arose which took many builds and flashes to diagnose and fix, given the nature of hardware. If we had designed our original datapath with these features in mind, it would have streamlined the development process significantly.

We gained mad respect for seniors who are able to do the same as us, but with arguably worse technologies. Honestly, from day 1, we knew that the journey in 50.002 will be hard, hearing all our seniors complain about it being the 'killer' mod, cautioning us to make sure that we were preemptively form a group with our close, reliable friends so that we would brave through the storm together. But we were severely underprepared; every step of the way was just fraught with difficulties. Circuits? No one in our group had done circuits before, the guy who completed it in the end, was literally watching a video on what a breadboard was the day before we started the circuitry and successfully got himself a shock metaphorically and literally from the circuit. Hardware skills? Basically none, honestly at this point, the FABLAB staff are the hidden 8th member of our group with the amount of invaluable advice that they have given to our group. On our first day in the FabLab, we spent a whole hour wondering why our LED matrix was not lighting up, even though it was lighting up just a moment ago. Ben just casually strolled over and

within 10 mins, figured out that the wire connecting the LED matrix to the power supply was dead.

If we were to condense our experience into 3 points to tell ourselves before we started on the project, the first would definitely be to never trust your hardware. In software, a bug would likely be a logic error or just a misplaced colon, but in hardware, it can be anything and everything, a dead wire, loose connections or just not turning on your power supply. It almost seems like hardware is the world's way of getting back at us. The second would be something that the school preaches a lot about, which is designing for your intended user. Just because the game was easier for our team does not mean that the difficulty has decreased, but rather we have just become more skilled at it. We can just imagine what would be a 'Stardew Valley' for us would be an 'Elden Ring' for the 5 year old that may play it at the showcase. Lastly, it would be to not assume and ask. We spent way too much time assuming what errors we have made when there are literal hardware experts that likely have 10+ years of experience over us.

To end off, if we were to describe this journey, we would name it "Firsts". For most of our group, they experience a first for many of the tasks they do, Be it the first time lasercutting, the first time designing a game or the first time, getting shocked by electricity, it definitely made a lot of memorable experiences for us. We have been constantly pushed to our absolute limits and felt quite uncomfortable dealing with a very unfamiliar domain, but we have definitely gained a lot from it. We feel that a few years down the road, it would definitely become a memory we look fondly upon. Honestly, thinking about it, it is kind of reminiscent of NS, really tough and defeating when you go through it, but rewarding after you have overcome it. And like NS, most of our group may not wish to take the course again.

References

- Agus, N. (2024). *FPGA 3: 7-Segment Display*. 50.002 Course Notes. Retrieved from https://natalieagus.github.io/50002/fpga/fpga_3_2024. (Referred to diagrams for the mapping of pins to segments).
- Agus, N. (2024). *WS2812B LED Strip*. 50.002 Course Notes. Retrieved from <https://natalieagus.github.io/50002/fpga/ws2812b> (Adapted the WS2812B driver module for the project).
- Original pixel art assets were created for this project. Visual inspiration for the art style and color palettes was drawn from general online image searches of classic 8-bit and 16-bit video games

Appendix

1. ALU Design and Tests

For the ALU presented during Checkoff 1, see [ALU \(Checkoff 1\)](#)

For the ALU used in the game itself (which has undergone further changes), see [ALU \(1D\)](#)

<Tests: Correlation between input stimuli, such as button presses or other inputs, and the anticipated corresponding outputs expected from the prototype>

Focus	Test	Expected output	Actual output	Pass?
Menu	Move left and right buttons pressed on difficulty and weapon selection screens	Difficulty / weapon screen changes	Chosen difficulty / weapon screen changes	YES
	Move up, down or attack buttons pressed on weapon selection screen	Nothing happens	Nothing happened	YES
	Pick HARD difficulty, pick BLAST weapon, trigger game over and return to difficulty + weapon selection Repeat test for different difficulties and weapons	Game remembers the previously chosen weapon and difficulty	HARD and BLAST automatically selected	YES
Enemy	Player starts game and does nothing (observe enemies spawning) Repeat test on EASY, MEDIUM and HARD	Enemies spawn in random tiles on the edge at a regular rate based on the difficulty and move towards the player Game over screen appears when enemies touch the player	Enemies only spawn on the edge Enemy spawns appear random Enemies move towards the player Difficulty affects spawn rate as programmed Game over screen appeared	YES
Player	Player moves into	Player's ammo increases	Player's ammo	YES

	ammo pickup with 10 ammo	from 10 to 12 (capped) Ammo pickup immediately respawns in a random location	increased from 10 to 12 and ammo pickup respawned immediately in a random location	
	Move left button pressed while player is on the left edge Repeat test on all edges	Player does not move	Nothing happened (player cannot move)	YES
	Player moves into an enemy	Game over screen appears	Game over screen appeared	YES
	Player attacks repeatedly	Player color changes from blue to white	Player color changes from blue to white	YES
Attack	Player attacks enemies using LASER	Correct shape and color drawn Enemies in range eliminated Points increases for every enemy eliminated Shape disappears after a split second	Three cyan lines drawn Enemies in range eliminated Points increased for every enemy eliminated Shape disappeared	YES
	Player attacks enemies using BLAST	Correct shape and color drawn Enemies in range eliminated Points increases for every enemy eliminated Player moves back Shape disappears after a split second	Orange cone shape drawn Player moved back Enemies in range eliminated Points increased for every enemy eliminated Shape disappeared	YES
	Player attacks enemies using SLASH	Correct shape and color drawn Enemies in range eliminated Points increases for every enemy eliminated Player moves forward Shape disappears after a split second	Purple semicircle shape drawn Enemies in range eliminated Points increased for every enemy eliminated Player moved forward Shape disappeared	YES

	Player attacks with 0 ammo	Player cannot attack	Nothing happens (player cannot attack)	YES
Win	Player survives for 60 seconds	Win screen appears	Win screen appeared	YES
Edge cases	Press all 4 move buttons + all 4 attack buttons at the same time	Game handles all 8 inputs as quickly as possible with no input lost	Player moved in all four directions and attacked in all four directions once	YES

2. Prototype code + Repo link

<https://github.com/50002-computation-structures/2026-1d-project-no-1-designai-team/tree/main>

3. Project Management Log: Team Tasks

Member	Task
Chan Yong Ze	Hardware: Design and assembled physical enclosure
Kwok Zhan Rui	Hardware: Designing and connecting the circuitry
Lin Chao Ran	Hardware: Procured and stored logistics
Ong Yu Hang	Code: Inspired architectural changes to make a more efficient game
Rajeev Bharath	Code: Balanced gameplay difficulty
Yip Jing Han	Code: Designed the core gameplay loop and code
Zhang Xirui Asher	Code: Managed and coordinated tasks between the group members, preventing conflicts

4. Components' Specifications

- WS2812B:
 - Type: addressable RGB LED

- Operating voltage: 5 V
 - Data signal: digital, single-wire
 - Color depth: 24-bit color, 8 bits each for R, G, B
 - RGB range: 0–255 for each color
 - Number of pins: VCC, GND, DIN, DOUT
- 7 Segments
 - Type: common anode
 - Number of digits: 4-digit
 - Operating voltage: 1.8–2.3 V per segment
 - Segment color: Red
 - Input: segment control signals for a, b, c, d, e, f, g
 - Display values: digits 0–9
- Buttons:
 - Type: momentary push-button
 - States: pressed / not pressed
 - Signal: digital input, 0/1
 - Number of pins: 4-pin tactile switch
- Power supply:
 - Output voltage: 5 V
 - Maximum current: 48 A
 - Type: DC adapter
 - Regulated via surge protector on plug
 - Powers LED matrix